

SpinTrace: T4 Project Plan

Oct 6, 2026

Start here tonight

Every step from now until you sleep, in order. Watch the first crawl yourself before setting the goal: that is the step that cannot go wrong overnight.

1. Set up the repo (about 15 minutes)

- ☐ Create a private GitHub repo
- ☐ Upload `CLAUDE.md`, `PLAN.md` and the assignment PDF (named `CSD358_IR_Mid-term_Assignment_2026.pdf`) with Add file, Upload files
- ☐ Upload `Team_plan.pdf` into `docs/` so the diagrams are in the repo too
- ☐ Create `.claude/settings.json` with Add file, Create new file, typing the full path as the name, then paste the reviewed rules
- ☐ Commit with a message like "Add planning docs (written 12:00 to 21:00, before the build)"
- ☐ Message Teammate A: build the seed list (originals plus suspected farms) and verify three NewsGuard Wayback pairs, both into the repo by 22:00

2. Choose where Claude Code runs

The first cloud session could reach only PyPI: news sites, Wikinews and the Wayback Machine were blocked, so it could not crawl.

- ☐ Cloud: open the environment settings for this repo and switch network access to full internet, if that option exists
- ☐ In a new cloud session, first prompt: "Before anything else, check that you can reach `bbc.com/robots.txt`, `en.wikinews.org` and `web.archive.org`, and report the results."
- ☐ If all three load, stay in the cloud: the laptop does not need to stay awake
- ☐ If there is no full-access option or the check fails, go local: clone the repo, run `claude` in the folder, and follow the laptop items under Overnight run

3. Start the build, awake

- ☐ Switch to auto mode and turn on waiting for usage limits to reset
- ☐ Accept the workspace trust dialog after reading the rules it lists; confirm with `/permissions`
- ☐ First prompt: "Read `CLAUDE.md`, `PLAN.md` and the PDF. Propose a short design, then build the crawler first and start it in tmux, logging to a file."

- ☐ Watch the crawl for a few minutes: disallowed pages skipped, delays between requests to each site, articles saved
- ☐ If anything is blocked or the agent starts substituting data, send: "Pause. The live crawl is a non-negotiable: don't substitute synthetic or bundled data for crawled pages. If a host is blocked, log it in PROGRESS.md and tell me instead of working around it."

4. Hand over and sleep

- ☐ Set the suggested goal from Overnight run, with your wake-up time filled in
- ☐ Cap spending on any API key used for rewrites; if local, keep the laptop plugged in, awake and lid open

5. In the morning

- ☐ Run `/goal` for its status and last reason, then read PROGRESS.md, the git log and the crawl log
- ☐ Start judging flagged pairs with Teammate A

Overview and scale decision

Build a deep vertical slice: every stage working end to end at modest scale, the novel detection stage built thoroughly, on an architecture that grows into the course project without a rewrite.

SpinTrace in one line: a polite, focused crawler that catches news articles rewritten from other outlets, including AI paraphrases that standard duplicate checks miss, and traces each one back to its original.

Option	What it means	Effect on marks	Verdict
Thin prototype	Detection on a static dataset, with no crawler or a toy crawl of a few hundred pages	Weak track relevance; corpus too small for ranking to matter	No
Vertical slice	Polite crawl of 15 to 25 sites (several thousand pages), full detection pipeline, ranked search, rigorous evaluation	Maximises IR principles (30), working system (20) and evaluation (15)	Build this
Full scale	Distributed crawling, freshness scheduling, dashboard, very large corpus	No marks for scale; burns hours evaluation needs; more demo risk	Course project

The hackathon build is Milestone 1 and must stand alone as a complete submission. This tab is for the two of you; what Claude Code builds, in what order and how it is evaluated lives in `Agent brief` , which leaves implementation choices open.

Novelty claim and positioning

Claim: SpinTrace extends the crawler’s “content seen?” check from copy detection to paraphrase-aware derivative detection: it retrieves a rewrite’s likely source using the facts a rewrite rarely changes, then traces provenance and demotes copies in ranking.

The insight that makes it IR, not just NLP. A rewrite changes wording but keeps facts: names, numbers, dates and direct quotes, which are often copied verbatim. Those are high-idf terms, so a suspect article’s rare terms become a query against the inverted index (index elimination), and its quotes become phrase queries on a positional index. IR finds the candidates; semantic similarity only verifies them.

Why it matters. In NewsGuard’s test, Grammarly’s plagiarism checker could not find the source of 34 of 43 AI-rewritten articles, a 79% failure rate ([NewsGuard, Aug 2023](#)). NewsGuard now tracks 3,749 AI content farm sites, and its full list is not public ([AI Tracking Center](#)).

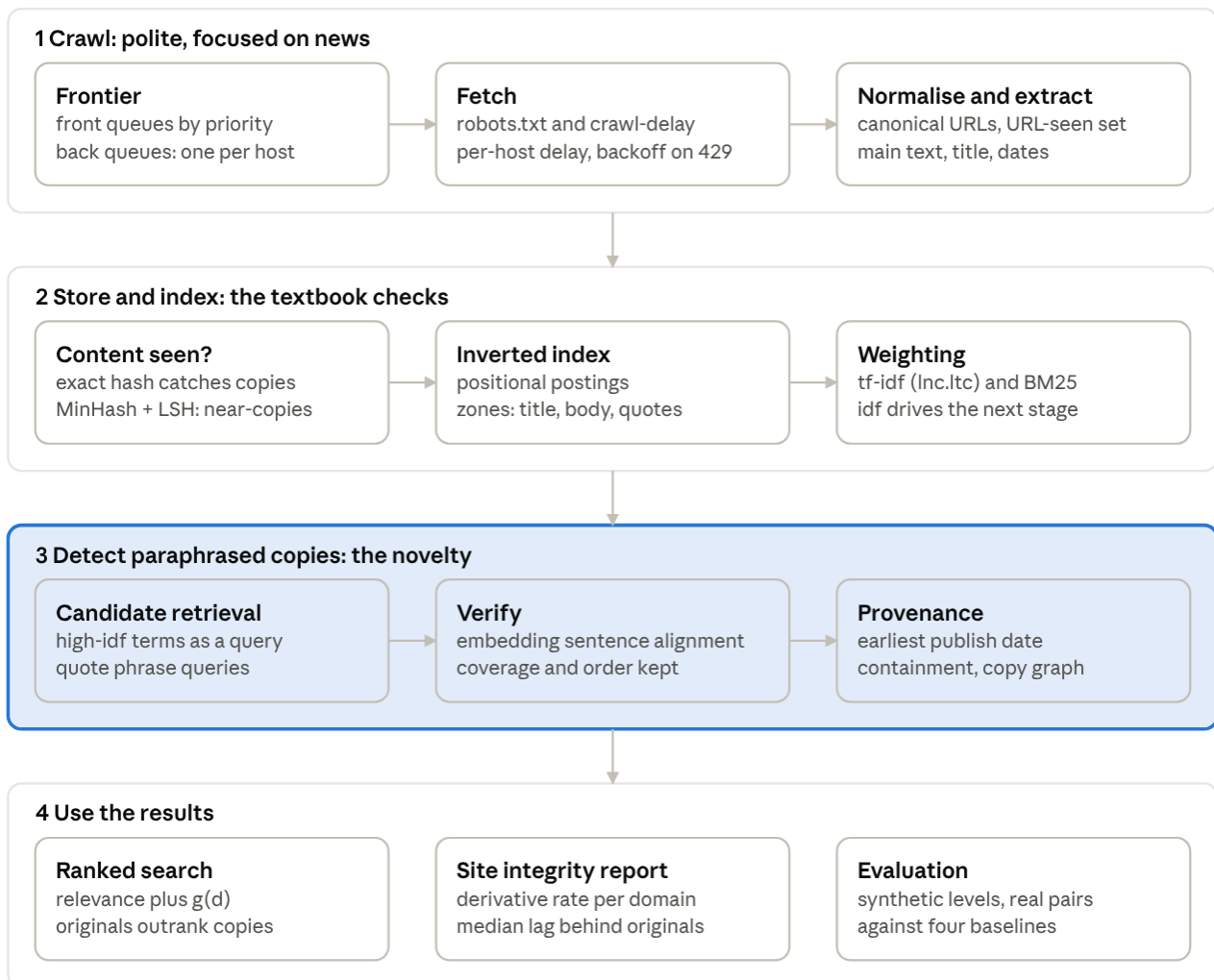
Existing approach	What it catches	Why it fails on AI rewrites
Exact hashing (textbook “content seen?”)	Identical copies	Any edit breaks the hash
Shingles with MinHash or SimHash	Near-copies and light edits	Paraphrases share few shingles
Plagiarism checkers	Verbatim reuse	Missed 79% of sources in NewsGuard’s test
Domain blocklists (browser extensions, GitHub lists)	Known farm domains	Manual and stale; new domains slip through
AI-text detectors	Text that looks machine-written	Unreliable, and never name the source
SpinTrace	Rewrites of any intensity, plus their original	Weak spots measured in the evaluation

A GitHub API search (Oct 2026) found blocklists and generic plagiarism checkers, but no project combining a crawler, paraphrase-aware detection and provenance tracing.

System architecture

Four stages run in order; only stage 3 is new, and it reuses stage 2's index and idf weights to find candidates before any embedding model runs.

IR finds the candidates; semantic alignment only verifies them



SpinTrace pipeline · 4 stages, 12 components

Exact hashing and MinHash stay in the pipeline as the textbook "content seen?" check, so they double as the baselines stage 3 must beat.

Derivative score: a weighted mix of alignment coverage, order preservation, and quote and entity overlap, with weights and threshold tuned on the dev split.

Ranking: each document's static quality $g(d)$ is its originality, near 1 for originals and near 0 for confirmed copies.

$$\text{net score}(q, d) = \text{relevance}(q, d) + \lambda \cdot g(d)$$

Data sources and ethics

Three real sources plus one synthetic set: a live crawl of originals and suspected farms, documented NewsGuard pairs, and graded rewrites generated from freely licensed news.

Source	Role	Notes
Live crawl: original outlets (e.g. The Guardian, BBC, NPR, The Conversation, Wired, The Verge, Indian outlets)	Originals for the index and the demo	Check each robots.txt at hour 0; discover articles via RSS and news sitemaps
Live crawl: suspected farm sites	Where real rewrites appear	Domains named in NewsGuard's report if still live, plus new ones from manual farm hunting
NewsGuard-documented pairs	Real, third-party-verified ground truth for the reveal	Farm copies via Wayback snapshots; prefer pairs with open originals (Wired, The Verge, The Guardian, Bored Panda)
Wikinews dump	Originals for synthetic rewrites	Freely licensed (CC BY), so rewrites can be shared in the repo
LLM-generated rewrites	Labelled evaluation set at graded difficulty	Declared in the AI-use section
CC-NEWS, PAN, METER (optional)	Scale and external benchmarks	Check access and formats before relying on them

Farm hunting is manual research, not automated search scraping: a teammate searches for chatbot error strings such as "As an AI language model" on news-style sites, verifies each hit, and adds the domain to the seed list.

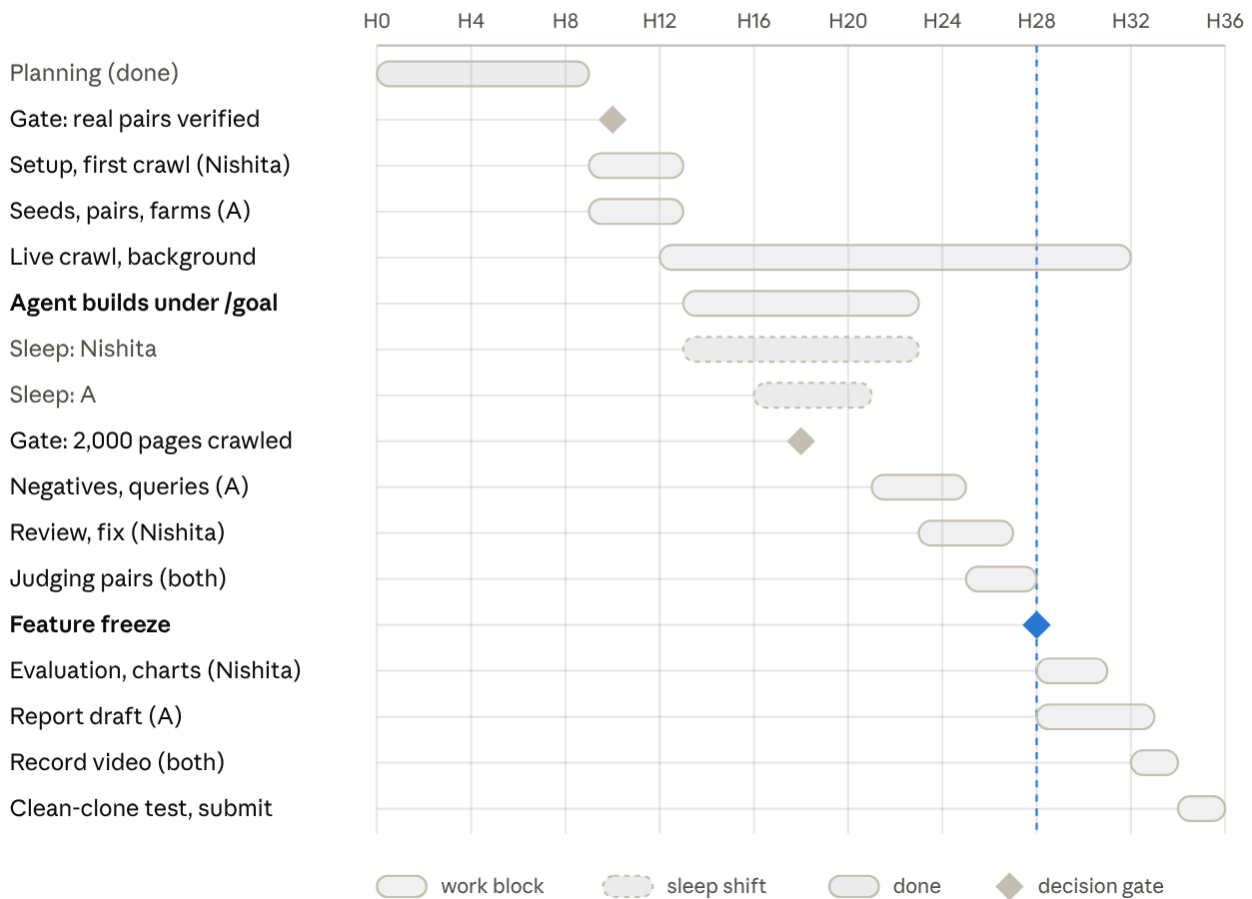
Ethics rules (robots.txt, delays, no personal data, no crawled texts in the repo) are enforced in code; they are listed in [Agent brief](#).

36-hour timeline

Hours count from the track announcement (hour 0 = 12:00); planning used hours 0 to 9, so the build runs 27 hours from 21:00. The crawler is built first so it collects pages in the background for about 18 hours.

Claude Code builds overnight while you sleep; features freeze at hour 28

Hours since tracks were announced (H0 = 12:00); planning done at H9 (21:00); owners in brackets



Build timeline for two · 27 hours from 21:00, 2 sleep shifts, 3 gates

Gates:

- Hour 10 (22:00), real pairs verified:** at least three NewsGuard pairs load from the Wayback Machine and every seed's robots.txt is checked. If not, swap in other documented pairs or seeds before building further.
- Hour 18 (06:00), 2,000 pages crawled:** if the live crawl is short, ingest one CC-NEWS file so ranking still matters.
- Hour 28 (16:00 tomorrow), feature freeze:** after this, only bug fixes, evaluation runs, writing and recording.

The live crawl runs unattended during the first sleep shift, so it logs errors to a file and stops itself if a host returns repeated 429s.

Overnight run

`/goal` keeps Claude Code taking turns without you, but on its own it stalls: it does not change the permission mode, and it pauses on usage limits ([Claude Code docs](#)). Set up all of this before you sleep.

- ☐ Start the session in auto mode, so tool calls run without approval prompts; in Manual mode the first unapproved command stops the run
- ☐ Turn on waiting for usage limits to reset, so a usage-limit pause resumes by itself instead of waiting for you
- ☐ Run the live crawl as its own process (tmux or nohup) writing to a log, not as Claude's background task; background work defers goal checks, and idle check-ins stop after three
- ☐ Keep the laptop plugged in, awake and online: disable sleep (`caffeinate -i` on macOS), pause OS updates, keep the lid open
- ☐ Put the agent brief in the repo so it survives context compaction, then commit and push
- ☐ Cap spending on any LLM API used for rewrite generation

Pre-approved permissions: put the reviewed allow and deny rules in

`.claude/settings.json` before starting. Claude Code lists them in the workspace trust dialog, and `/permissions` shows them any time. File edits approved at a prompt only last for the session, so the `Edit` rule has to be in the file.

Paste this as `.claude/settings.json`. Allowing `python *` lets Claude run any Python, which the project needs, so keep nothing sensitive in the folder; deny rules stop the usual mistakes but are not a security boundary.

```
{
  "permissions": {
    "allow": [
      "Edit(/**)",
      "Bash(python *)",
      "Bash(python3 *)",
      "Bash(pip install *)",
      "Bash(pip3 install *)",
      "Bash(pytest *)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git push *)",
      "Bash(git checkout *)",
      "Bash(git branch *)",
      "Bash(git stash *)",
      "Bash(tmux *)",
```

```

    "Bash(mkdir *)",
    "Bash(touch *)",
    "Bash(mv *)",
    "Bash(cp *)",
    "Bash(sqlite3 *)"
  ],
  "deny": [
    "Bash(git push --force *)",
    "Bash(git push -f *)",
    "Bash(git reset --hard *)",
    "Bash(git clean *)",
    "Bash(rm -rf *)",
    "Bash(sudo *)",
    "Read(./env)"
  ]
}

```

Suggested goal. The evaluator only reads the transcript, so the condition names checks Claude must run in it:

```

/goal The "end to end first" priorities in the agent brief all run on real
crawled data: the README's smoke-test command exits 0, the test suite passes,
the synthetic evaluation has written its results table and F1-by-level chart,
PROGRESS.md lists what is done, what failed and what is next, and all work is
committed. Never change the brief's non-negotiables. Or stop at <your wake-up
time>.

```

In the morning: run `/goal` for its status and last reason, read PROGRESS.md and the git log, and check the crawl log for 429s and robots.txt skips. A goal can also stop early with a no-progress warning or after an unrecoverable error (auth, credits, context overflow); the reason is in the transcript.

Work division

Nishita drives Claude Code with the agent brief; Teammate A verifies, runs and owns the work that needs human judgment. Each explains their own components in the video.

Member	Owens and explains	Main work
Nishita	Detection pipeline, index, evaluation	Drives Claude Code and reviews every module; runs experiments and charts; README

Member	Owns and explains	Main work
Teammate A	Crawler and search	Tests the crawler and monitors the live crawl; seed list and farm hunting; verifies NewsGuard pairs; collects hard negatives; judged search queries; drafts the report and edits the video

Shared duties: both of you judge every flagged live pair independently, so agreement can be reported; both keep the AI-use log current.

Video plan

Target 7 minutes, no slides: every segment is a live terminal, notebook or browser, and each member explains the component they own in their own segment.

Time	On screen	Speaker	Required element
0:00 to 0:50	A NewsGuard pair side by side in the browser; the 79% plagiarism-checker failure; "a crawler built to catch crawlers"	Nishita	Problem and track fit, under 1 minute
0:50 to 2:15	Crawl starting on seeds: frontier queues, a robots.txt skip, per-host delays in the log	Teammate A	System running; crawler explained by its owner
2:15 to 3:45	The reveal on a real pair: low Jaccard, candidates from rare terms and a quote match, alignment, provenance verdict, intermediate output printed	Nishita	Pipeline with intermediate output
3:45 to 4:15	A limitation, live: a same-event false positive from the hard negatives	Teammate A	At least one limitation
4:15 to 5:00	One query with and without g(d): copies drop below their originals	Teammate A	Search explained by its owner
5:00 to 6:20	F1-by-level chart, source retrieval, ablation, real-world catches, all against baselines	Nishita	Evaluation results vs baseline

Time	On screen	Speaker	Required element
6:20 to 7:00	Limitations and next steps, back in the terminal	Both	Close

Recording rules: crawl and index beforehand so live runs take seconds, but every command really runs on camera; screenshots alone score 0. Use a large terminal font and record each segment separately.

Report plan

Eight pages, following the brief's seven sections, with the IR section and evaluation taking more than half the space because they carry 45 of the marks.

Section	Pages	What goes in
1. Problem and track relevance	1	NewsGuard findings, who is harmed (publishers, readers, advertisers), why it is a WWW web-integrity problem, papers referred
2. How we used IR	2.5	Pipeline diagram; each concept with its file and the reason for choosing it: frontier, robots, normalisation, content-seen check, shingles and MinHash, inverted and positional index, zones, tf-idf and BM25, index elimination, g(d) and net score
3. Beyond IR	0.5	Sentence embeddings, NER, LLM rewrite generation, and how each serves an IR stage
4. Novelty	0.75	The claim, the comparison table, the facts-survive-rewriting insight
5. Evaluation	2	E1 to E5 tables and charts, ablations, honest losses
6. Limitations and next steps	0.75	Known weak spots, plus the roadmap if continuing
7. Work division and AI use	0.5	Ownership table; AI-use declaration from the shared log

References to cite (from memory; verify each before submitting):

- Broder (1997), on the resemblance and containment of documents: shingling and containment
- Charikar (2002): SimHash

- Manku, Jain and Das Sarma (2007), detecting near-duplicates for web crawling, WWW
- Heydon and Najork (1999), Mercator: a scalable, extensible web crawler
- Robertson and Zaragoza (2009), the probabilistic relevance framework: BM25 and beyond
- Reimers and Gurevych (2019), Sentence-BERT
- Clough et al. (2002), METER: measuring text reuse in journalism
- Potthast et al., PAN shared-task overviews on plagiarism source retrieval
- Manning, Raghavan and Schütze (2008), Introduction to Information Retrieval, chapters 19 and 20
- [NewsGuard \(2023\)](#) and the [NewsGuard AI Tracking Center](#)

Risks and mitigations

No single failure sinks the project: the detection pipeline and evaluation never depend on the live crawl, so the worst case is a weaker live demo with strong offline results.

Risk	Likelihood	Mitigation	Fallback
Same-event coverage flagged as copying	High	Hard negatives in evaluation; require alignment and order, not just shared names	Report the false-positive rate honestly
Scope creep	High	Priorities in the agent brief; feature freeze at hour 28	Cut the "only if ahead" items first
Exhaustion: two people, 36 hours	High	Staggered sleep: A sleeps while the crawl runs, Nishita while A judges and tests search	Shorter critique loop
Crawler slow or blocked	Medium	RSS and sitemaps first; drop blocked seeds; crawl runs in the background from hour 14	Under 2,000 pages by hour 18: ingest one CC-NEWS file
No live farm rewrites found	Medium	NewsGuard pairs and the synthetic set carry evaluation	The reveal uses a NewsGuard pair
Rewrite generation slow or rate-limited	Medium	Start early; cache every output	Drop the translation level

Risk	Likelihood	Mitigation	Fallback
Unreliable publish dates	Medium	Prefer original-outlet metadata, then crawl time, then Wayback first capture	Mark provenance "uncertain"
Teammate A has no Claude Code	Medium	He owns testing, running, judging and writing, which need no agent	Nishita generates, he verifies
Wayback snapshots fail to load	Low	Verify three pairs in the first two hours	Swap in other documented pairs

Course project roadmap

If you continue, each milestone adds one stage to the hackathon architecture; none requires rebuilding what exists. If you don't, this list becomes the report's "next steps".

1. **Freshness and scale:** a recrawl scheduler that estimates each page's change rate (Cho and Garcia-Molina), full Mercator front and back queues, multi-process crawling, and continuous CC-NEWS ingestion.
2. **Farm-aware focused crawling:** the frontier prioritises domains with high derivative rates and discovers new farms through shared page templates and link neighbourhoods.
3. **Cross-lingual derivatives:** catch English originals rewritten into Hindi, a likely pattern for Indian farms, using cross-lingual retrieval and alignment.
4. **Learned scoring:** learning-to-rank over the detection signals, replacing hand-tuned weights.
5. **A measurement study:** how fast farms copy, which outlets they target, and how long copies outrank originals.
6. **Product:** a browser extension that marks an article as a rewrite and links its original, plus alerts for publishers.